Introduction to Design Patterns (Part 3)

Behavioural patterns — Advanced Programming

Ayoub FRIHAOUI

ayoub.frihaoui@lecnam.net
Conservatoire national des arts et métiers (CNAM Paris)

Original material © Rezak AZIZ, CNAM Paris.
Source: `https://rezakaziz.github.io/PROG-AVANC/`

Behavioural patterns

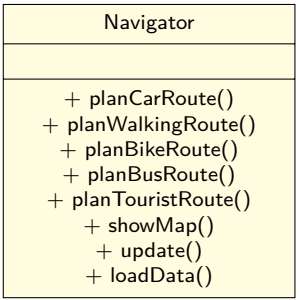
Section 1

Strategy

Strategy — Problem

Problem

A single `Navigator` class contains several variants of the same algorithm: car, walking, bike, bus routes, etc. This class becomes huge, hard to maintain and very fragile.



Problems: - Giant class
- Algorithm multiplication
- Strong coupling and duplication
- Fragile code at the slightest change
- Team conflicts (Git)

Strategy — Intent

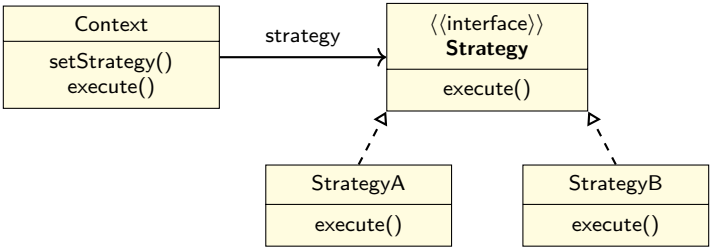
General intent

Define a family of **interchangeable algorithms**, encapsulate them in independent classes, and let the client **dynamically change** the algorithm used by the context.

What we want to avoid

Long conditional structures: `if (mode == car) ...  else if (mode == bike) ...`

Strategy — Structure



Strategy — Applicability

When to use it?

- Several variants of one algorithm (sort, compression, AI...).
- Dynamically changeable behaviour (game, UI, math).
- Replace a big block of conditions with polymorphism.
- Isolation of complex algorithms.
- Respect the Open / Close principle.

Strategy — Advantages and drawbacks

Advantages

- + Algorithm interchangeable at execution.
- + Fewer complex conditions.
- + Easy to add new behaviours.
- + Simplified unit tests.
- + Strongly decoupled.

Drawbacks

- - Class proliferation.
- - The client must know the right strategy.
- - May feel heavy for small algorithms.
- - Can be replaced by lambdas in some modern languages.

Section 2

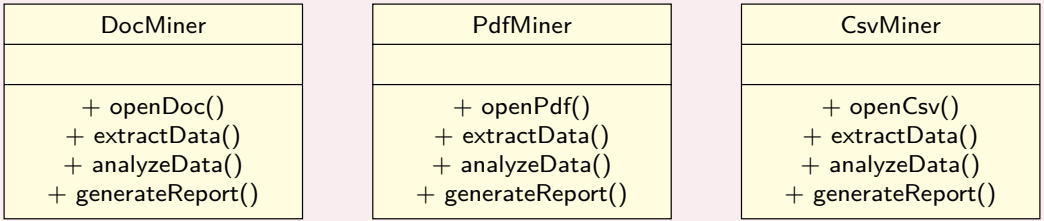
Template Method

Problem to solve

Observation

In many applications, several classes implement an **almost identical** algorithm, with only a few differences.

Example: Data Mining (PDF, DOC, CSV)



Problems

Almost identical algorithms $\rightarrow$ strong code duplication.

Intent of the pattern

Main goal

Define the **skeleton of an algorithm** in a parent class, while leaving subclasses the possibility of **redefining some steps**.

Key idea: we fix the *sequence* of operations, but let their *content* vary.

What we avoid

```
if (format == PDF) ...  else if (format == CSV) ...
```

Template Method — Solution

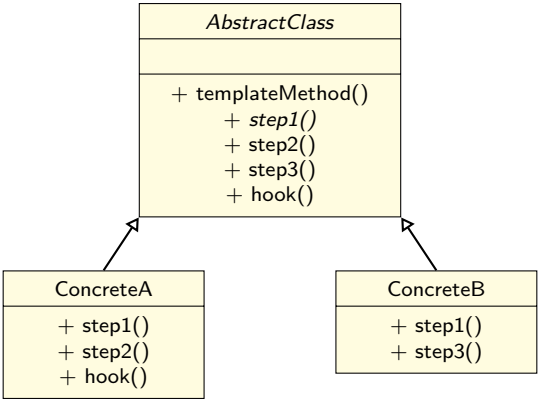
Principle

- Break the algorithm into steps.
- Place the chaining of these steps in a **final** method: the template method.
- Declare some steps as:
  - abstract → implemented by subclasses;
  - optional → with a default implementation;
  - hooks → empty implementation, can be redefined.

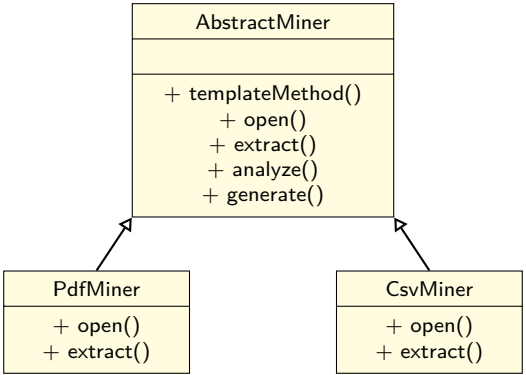
Consequence

The structure of the algorithm is **fixed**, but its steps may vary with the subclasses.

Template Method — Structure



Structure of the Template Method



Template Method — Applications

When to use this pattern?

- When an algorithm is identical in several classes, with only a few variations.
- When you want:
  - **share code** between subclasses;
  - **force an execution order**;
  - provide **controlled extension points**.
- When a client wants to extend only some steps.
- When you want to replace a big monolithic algorithm with well-defined steps.

Advantages

+ Factored code

Common steps are in the parent class.

+ Stable structure

The algorithm is clear and fixed.

+ Controlled extensibility

Subclasses customise only what must vary.

+ Polymorphism

The client no longer needs if/else or switch.

Drawbacks

- Rigid structure

Impossible to modify the order of steps.

- Risk of LSP violation

If a “default” step makes no sense for a subclass.

- Can become complex

Many steps $\rightarrow$ abstract class hard to maintain.

Section 3

Visitor

Problem — Context

Situation

We develop an object-oriented application handling a **complex structure of objects**.

- Objects are organised as:
  - tree
  - graph
  - composite structure
- Each object represents a different entity

A geographic-data application.



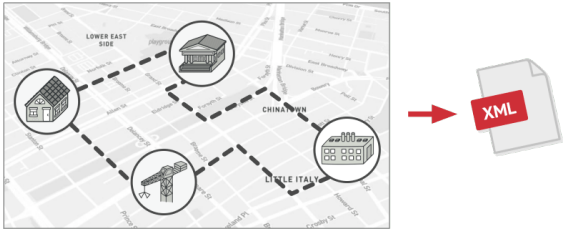
- Objects can be:
  - a City
  - an Industry
  - a TouristSite
- All these objects are linked together

Problem — New need

New request

Implement exporting the graph in **XML** format.

- The graph is made of **several object types** (city, industry, tourist site, etc.).
- Each object type must be exported **differently**.



What solutions do you propose to solve this problem?

Naive solution no. 1

Idea

Add an `exportXML()` method in each class.

- `City.exportXML()`
- `Industry.exportXML()`
- `TouristSite.exportXML()`

Problems

- Node classes are already in **production**
- The architect forbids any modification
  - High risk of introducing **bugs**
  - The XML export code adds a responsibility to the classes (SRP violation).
  - Each new format (JSON, CSV,...) would mean **modifying all classes**

Naive solution no. 2

Idea

Create an **external class** dedicated to XML export. Walk the graph from that class. Handle each node type separately.

Conditional approach

```
if (obj instanceof City) { ... }
else if (obj instanceof Industry) { ... }
```

Problems

- The export class must know **all node types**
- Using if/else or switch on types
- Code strongly **coupled** to concrete classes
- **Open/Closed** principle violation

Constraints of the problem

What we want

- Add new behaviours
- Avoid `if/else`
- Respect polymorphism

What we refuse

- Modify existing classes
- Couple the code to concrete types
- Multiply responsibilities

Intent of the Visitor pattern

Goal

Separate the **algorithms** from the **objects** they operate on.

Principle

- Node classes **unchanged**
- Treatments moved into **visitors**
- Add operations without modifying objects

Problem solved

- Code already in production
- Modifications forbidden
- Avoid `if/else`

Usage examples

- XML export
- JSON export
- Statistics

How the Visitor solves the problem

Key idea

The Visitor pattern relies on a technique called **double dispatch**, which calls the right method without conditional blocks.

Principle

- The client no longer chooses which method to call
- The decision is delegated to the objects themselves
- Each node knows its own class
- So it chooses the matching method of the visitor

Mechanism: `accept(visitor)`

- The node **accepts** a visitor
- It calls the visitor method matching its type

Single dispatch vs Double dispatch

Single dispatch (classical case)

- Default mechanism in OOP
- The called method depends on the type of the receiver object

`obj.method(param)`

- The chosen method depends on `obj`

Double dispatch (Visitor)

- The called method depends:
  - on the node type
  - on the visitor type
- Two successive calls

`node.accept(visitor)`

- 1st dispatch: on the node type
- 2nd dispatch: on the visitor type

Key message

Double dispatch chooses a method based on **two dynamic types**.

Double dispatch: code example

Client code

```
for (Node node : graph)
    node.accept(exportVisitor);
```

Node-side implementation

```
// City
class City {
    void accept(Visitor v) {
        v.doForCity(this);
    }
}
// Industry
class Industry {
    void accept(Visitor v) {
        v.doForIndustry(this);
    }
}
```

Important remark

Yes, node classes are slightly modified, but this modification is:

- minimal

Key consequence of the Visitor pattern

Generalisation

- All visitors implement a **Visitor interface**
- Nodes accept **any** visitor

Adding a new behaviour

- Create a new visitor class
- Implement the methods for each node type
- No change in existing classes

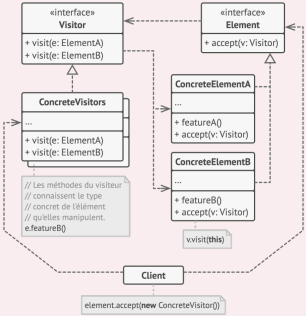
Key message

Stable structures, extensible behaviours

Structure of the Visitor pattern

Main elements

- **Visitor**
  - Visitor interface
  - One method per node type
- **ConcreteVisitor**
  - Implements treatments
  - (XML/JSON export, statistics, etc.)
- **Element**
  - Declares `accept()`
- **ConcreteElement**
  - Implements `accept(visitor)`



How it works

The client walks the structure, calls `accept(visitor)`; the node calls the right visitor method.

Applicability of the Visitor pattern

When to use Visitor

- The object structure is **stable**
- Behaviours evolve often
- Classes must not be modified
- Several treatments apply to the same objects

Typical contexts

- Graphs, trees, AST
- Composite structures
- Complex business models

Concrete examples

- Data export (XML, JSON, CSV)
- Structure validation
- Statistics computation
- Report generation

Warning

- Adding a new node type is costly
- All visitors must be updated

Advantages and drawbacks of the Visitor pattern

Advantages

- Add new behaviours without **modifying** existing classes
- Respect the **Open/Closed** principle
- No more if/else and switch
- Business code grouped in dedicated classes
- Eases adding complex treatments

Drawbacks

- Hard to add a **new node type**
- All visitors must be modified
- Strong coupling between visitors and visited classes
- Adds design complexity
- Unintuitive for beginners

To remember

Visitor works if the structure is **stable** and behaviours **evolve**

Thank you!